

Learn2Mask: A Predictive Encoding Framework for Energy-Efficient PCM Writes

1st Sullivan Gleason

Computer Science Department
Southeast Missouri State University
Cape Girardeau, Missouri
sullivanpgleason@gmail.com

2nd Prince Kakadiya

Computer Science Department
California State University, Northridge
Northridge, California
prince-shaileshbhai.kakadiya.285@my.csun.edu

3rd Marjan Asadinia

Computer Science Department
California State University, Northridge
Northridge, California
marjan.asadinia@csun.edu

Abstract—Phase-Change Memory (PCM) presents a compelling alternative to traditional DRAM due to its scalability and non-volatility. However, frequent bit-level changes during write operations pose challenges in energy efficiency, endurance, and reliability. This paper introduces Learn2Mask, a predictive encoding framework designed to minimize write-induced bit transitions in PCM. By leveraging a strided bitmasking scheme based on modular affine sequences, our approach encodes data using structured masks that reduce Hamming distance between memory states. While brute-force selection of optimal masks is computationally infeasible at runtime, Learn2Mask addresses this limitation by training lightweight machine learning models including logistic regression, XGBoost, and neural networks to approximate optimal encodings efficiently. Evaluations using data collected from the NVMain memory simulator demonstrate consistent reductions in write activity and energy cost across a range of configurations. Learn2Mask offers a scalable and intelligent solution for energy-aware and durable PCM system design.

Index Terms—Phase-Change Memory (PCM), Bit Flip Reduction, Strided Bitmask Encoding, XGBoost, Data Encoding, Write Optimization.

I. INTRODUCTION

Phase-Change Memory (PCM) has emerged as a promising candidate to replace or augment traditional DRAM, offering advantages such as non-volatility, high density, and low standby power. However, PCM suffers from critical limitations in write endurance and energy efficiency due to the high cost of programming bits. Repeated bit-level changes not only accelerate cell degradation but also introduce reliability issues such as write-disturbance errors and thermal crosstalk. Therefore, reducing the number of bit transitions during write operations is essential to enhancing the overall durability and performance of PCM-based systems [1-5].

Conventional write optimization techniques focus on minimizing unnecessary writes or transforming the write payload to resemble the current memory state more closely. While these methods offer measurable benefits, many rely on fixed transformation rules or exhaustive searches that are either suboptimal or computationally prohibitive for real-time use [1-3]. Specifically, brute-force approaches to identifying optimal bitmask transformations become infeasible as word lengths and mask spaces grow, limiting their practical applicability.

This work was supported by the National Science Foundation (NSF).

In this paper, we introduce Learn2Mask, a predictive encoding framework designed to minimize bit flips in PCM writes through structured bitmask transformations and machine learning. Our approach defines a family of interpretable bitmasks using modular affine patterns, enabling the creation of a compact and expressive encoding dictionary. To overcome the high complexity of brute-force selection, we propose training lightweight predictive models such as logistic regression, gradient-boosted trees, and neural networks to approximate the optimal mask for any given write payload at runtime.

We evaluate our approach using data collected from the NVMain memory simulator, demonstrating consistent improvements in write efficiency across a variety of configurations. Learn2Mask provides a scalable, low-overhead encoding solution that enhances both energy efficiency and endurance in PCM systems, paving the way for more intelligent and adaptive memory architectures.

The remainder of this paper is organized as follows. Section II reviews related work on write energy optimization techniques, Section III presents the proposed Learn2Mask framework, detailing the structured bitmask design and predictive modeling approach. Section IV describes the experimental setup and data collection process using NVMain. Section V discusses the evaluation results and analyzes trade-offs. Section VI concludes the paper.

II. RELATED WORK

Recent studies have extensively explored optimization techniques in Non-Volatile Memory (NVM), especially Phase Change Memory (PCM) using various machine learning techniques. Lim et al. [1] introduced a novel workload-aware and optimized write cycle management system for NVRAM. Their approach focused on improving endurance and performance by dynamically adjusting write strategies based on workload characteristics, which helps to distribute write operations more evenly across memory cells, thereby prolonging the lifespan of the memory and enhancing overall performance. Another significant contribution presented in [2] demonstrates the effectiveness of power-aware reinforcement learning in managing memory power consumption. Their method leverages reinforcement learning algorithms to predict and adjust power states dynamically, thus significantly reducing energy usage

while maintaining performance. The proposed system learns from historical power consumption patterns and optimizes the power states accordingly, leading to substantial energy savings without compromising the performance of the memory system.

In [3], authors proposed a deep neural network model designed for accurate and efficient performance prediction in memory systems. Their model emphasizes the ability to handle complex memory access patterns and predict performance metrics with high accuracy. By utilizing a deep neural network, the model can capture intricate relationships within the memory access patterns, enabling more precise performance predictions and, consequently, better optimization strategies for memory management. The work by Chen et al. [4] highlighted the potential of distributed reinforcement learning for optimizing power consumption in large-scale memory systems. Their approach involves deploying reinforcement learning agents across a distributed memory architecture to collaboratively optimize power usage. This method not only improves power efficiency but also enhances computational performance by balancing the load and reducing bottlenecks in the memory system. The distributed nature of the solution ensures scalability and robustness, making it suitable for large-scale implementations.

Additionally, in [5], the authors present a technique to eliminate redundant writes, reducing energy use by up to 50% and doubling endurance. Their method shows minimal performance impact, enhancing PCM's reliability and efficiency for future systems. The modular reinforcement learning approach presented by Kim et al. [6] provides a flexible framework for integrating various reinforcement learning models to manage different aspects of memory systems. This approach allows for the simultaneous optimization of multiple parameters, such as power consumption, latency, and endurance, by utilizing specialized reinforcement learning agents for each task.

For modern memory systems, [7] introduces Memory Cocktail Therapy (MCT), which leverages machine learning techniques and architectural insights to optimize memory system performance. By employing gradient boosting and quadratic lasso models, MCT dynamically selects the best memory management policy based on runtime conditions. This approach has been shown to significantly improve instructions per cycle (IPC), extend memory system lifetime, and reduce overall energy consumption compared to static policies.

III. PROPOSED METHOD

This section presents the proposed methodology in detail. We begin by outlining the motivation for reducing bit-level transitions during PCM write operations, which directly impact energy consumption and memory endurance.

We then provide a comprehensive overview of our encoding framework, highlighting its structured design based on stride-based bitmasking. Following this, we discuss the computational limitations associated with exhaustive brute-force search for optimal encodings, particularly as the encoding space grows. To address this, we introduce a learning-based approximation approach that leverages pre-trained machine

learning models to predict near-optimal encodings efficiently at runtime.

A. Motivation

PCM write operations are inherently costly in terms of both energy efficiency and endurance. Excessive bit-level changes during writes can accelerate cell wear-out and give rise to reliability issues such as write-disturbance errors [8], [9], thermal crosstalk [10], and premature device degradation [11], [12]. These challenges underscore the critical need to minimize bit-level transitions during PCM write operations.

To address this, several encoding techniques have been proposed. Yang et al. [13] introduced Data Comparison Write (DCW), a simple yet effective strategy that performs a write only when the incoming data differs from the existing data in memory. Cho and Lee [14] advanced this idea by proposing Flip-N-Write, which conditionally inverts the write payload to further reduce bit-level differences. Seyedzadeh et al. [15] introduced coset encoding, where the data is mapped to one of several predefined codewords (cosets) that minimize high-energy transitions.

Building on these foundations, our approach explores the potential of a structured masking framework in which each bitmask is composed from a set of interpretable basis patterns defined by modular stride-based rules. This structured representation enables the integration of learned inference strategies, allowing for efficient, near-optimal bitmask selection in real time without relying on costly brute-force computations.

B. Stride-Based Multi-Bit Encoding

The encoding structure for an N -bit system in our method comprises N **XOR** masks, each defined by a modular affine sequence. These masks are stored in a dictionary, which we refer to as the *Encoding Scheme*. The selected encoding is embedded as a series of flag bits at the beginning of each data word.

The memory overhead introduced by this method is given by $\frac{N}{L}$, where L is the total length of the data word. It follows that $U = L - N$ bits are available for actual storage. The encoding scheme itself is only stored once, so its overhead is negligible.

We also note: In our encoding scheme, the N encoding bits are written without masking. As a result, these bits contribute additional transitions during a write. Under the assumption of uniform randomness, these contribute an expected $N/2$ bit updates per word, which we account for in all reported results.

We define a mask M using a tuple (s, o) , where s is the stride (modulus) and o is the offset. This creates a mask where every s^{th} bit is set, offset by o .

Formally:

$$M[i] = \begin{cases} 1 & \text{if } (i - o) \bmod s = 0 \\ 0 & \text{otherwise} \end{cases} \quad \text{for } i \in [0, U) \quad (1)$$

For simplicity, in our study, we adopt what we refer to as the *Standard Scheme* where the i^{th} bit of an N -bit system corresponds to the mask $(i, 0)$. Although this approach yields

strong empirical performance, alternative bases can produce modest improvements. We chose not to explore them in this work for the sake of clarity and reproducibility.

C. Optimal Composite Mask Selection

The optimal encoding is one that minimizes the Hamming distance between the data word stored in memory D_{mem} and the payload to be written D_{write} using a given encoding scheme. This minimization problem can be formalized as :

$$\begin{aligned} &\text{Minimize} \quad \sum \Delta \\ &\text{subject to} \quad \Delta = D_{\text{mem}} \oplus (D_{\text{write}} \oplus M_{\text{comp}}) \\ &\quad \quad \quad M_{\text{comp}} = \bigoplus_{i=1}^N c_i M_i, \quad c_i \in \{0, 1\} \end{aligned} \quad (2)$$

where $\{M_i\}_{i=1}^N$ are the basis masks defined in the encoding scheme, and $\mathbf{c} = (c_1, \dots, c_N)$ is the encoding vector for each data word.

D. Brute-Force Limitations

In our method, we iterate over all configurations of $\mathbf{c}$ to find the optimal encoding for a given write payload. This calculation becomes infeasible to do in real time as the encoding space grows exponentially (2^N) for increasing values of N . This necessitates alternative strategies for real-time encoding prediction.

E. Learning-Based Encoding Prediction

To enable efficient run-time encoding, we train a lightweight predictive model offline to approximate the optimal encoding based on training data. In our study, we compare Logistic Regression, XGBoost, and a Multi-Layer Perceptron model.

We recognize that occasionally the model may produce an incorrect encoding that results in more bit transitions than the original write payload. In such cases, if the masking basis includes the full inversion mask $(1, 0)$, we optionally enable a fallback mechanism by flipping the inversion bit.

Fig. 1 illustrates the Learn2Mask encoding workflow, highlighting the XOR-based difference calculation, predictive model inference, and final data encoding steps.

First, the bitwise XOR of D_{mem} and D_{write} is computed to obtain the difference Δ . This is then passed into the pretrained inference model, which predicts the optimal encoding vector $\mathbf{c}$ for a given encoding scheme. We then apply this encoding to D_{write} to produce D_{encoded} . Finally, $\mathbf{c}$ & D_{encoded} are written together in memory.

IV. EXPERIMENTAL SETUP

This section outlines our experimental methodology. We first detail the process of generating and organizing the data for model training and evaluation. We then describe the configurations used in our experiment, including model choice, encoding sizes, and word widths.

We then evaluate our proposed method, with DCW and Flip-N-Write as points of reference. Finally, we discuss the trade-offs between specific model choices and how they affect overall performance.

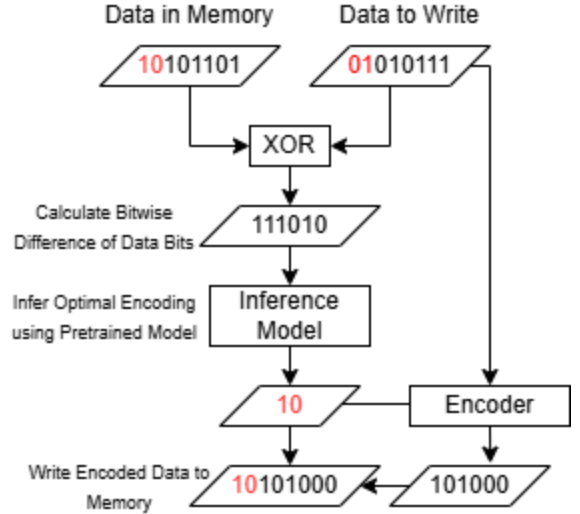


Fig. 1. Illustration of the Learn2Mask encoding process for a word length of 8 and encoding length of 2. The model predicts an optimal bitmask based on XOR difference, reducing bit-level write transitions.

A. Data Collection and Organization

We sourced data for this study from the NVMain simulator [16] [17], [18], [19], a respected and reliable simulator for non-volatile memory technologies. The data files were collected using real application workloads, such as those from SPEC CPU [20], and PARSEC benchmarks [21], to closely reflect realistic memory access patterns. By incorporating representative data access behaviors and diverse execution characteristics, these traces provide a faithful simulation of real-world scenarios, thereby improving the accuracy and reliability of our evaluations conducted with NVMain. It is also notable that NVMain is a versatile, cycle-accurate simulator that models both traditional DRAM and emerging NVM technologies such as PCM. It provides detailed insights into memory timing behavior, energy consumption, and NVM-specific characteristics like limited write endurance and multi-level cell structures. Additionally, NVMain supports hybrid memory architectures, fine-grained parallelism at the bank and subarray levels, and customizable memory controllers and address mapping schemes. Its ability to model key PCM features including asymmetric read/write latencies, high write energy, and endurance constraint makes it particularly suitable for evaluating PCM optimization strategies.

To simulate memory write behavior, we partitioned each data trace into two equal halves: one representing the current memory contents and the other representing the new write payload. We then applied a bit-wise XOR between the corresponding entries in the two halves to compute the bit-level transition data used for training and evaluation. For our experiments, we used 100,000 samples, with an 80/20 split between training and testing sets.

In our study, we tested encoding lengths of $N = \{1, 2, 4, 8\}$ bits and word lengths of $L = \{32, 64, 128, 256\}$ bits. Note that we treat $N = 1$ as equivalent to Flip-N-Write. Our results are

presented as a grid search across all combinations of encoding lengths and word lengths.

B. Generate Optimal Encodings

To generate training data for a given encoding length N and word length L , we extracted rows of length $U = L - N$ from our dataset and performed a brute-force search as outlined in (2). The result of this minimization process was used as the ground-truth label for each input and saved to a file for model training.

This procedure becomes computationally expensive for larger values of N , which limited our experiments to $N \leq 8$. Increased computational power, approximations or heuristics may allow this approach to scale beyond this limit. We leave the development of such methods to future work.

Fig. 2 visualizes the reduction in bit-level transitions across all tested combinations encoding lengths and word widths. Flip-N-Write and DCW are included in the figure as baselines. As expected, larger encoding sizes result in a decrease in average optimal bit updates, due to increased representational ability. All of our results account for $N/2$ bit updates per write from the encoding sequence and use the *Standard Scheme*, introduced in III-B.

Fig. 3 provides a more granular view by measuring the bit savings per encoding bit. Under the assumption that each encoding bit contributes, on average, 0.5 bit updates, this value serves as a baseline cost. All combinations of encoding lengths and word widths in our experiments achieve bit-efficiencies greater than this value, with a general upward trend in bit efficiency as word width increases.

Fig. 4 shows the percentage improvement over baseline strategies, including raw writes (no optimization), DCW, and Flip-N-Write. We also included the encoding overhead, helping contextualize how cost scales with coding complexity. Notably, improvements remain substantial even after accounting for this cost, indicating a favorable trade-off between compression effectiveness and overhead.

C. Model Training

For each combination of encoding lengths and data widths, we trained three inference models of varying complexity to evaluate their effectiveness at learning optimal encodings: Logistic Regression, Multi-Layer Perceptron (MLP), and XGBoost. Logistic regression was chosen for its simplicity and fast training time, MLP for its ability to learn complex nonlinear patterns, and XGBoost as a method to balance interpretability and performance.

The MLP was implemented as a fully connected feedforward network with three hidden layers with 128, 64, and 64 units respectively. All internal layers used ReLU activations. The output layer used a sigmoid activation to predict each encoding bit independently. We trained the MLP using binary cross-entropy loss and the Adam optimizer, with a learning rate of 0.001. Training was performed for 30 epochs with a batch size of 32.

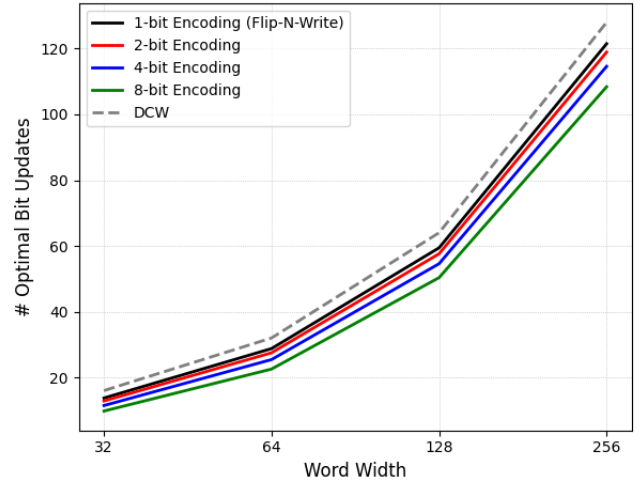


Fig. 2. Average number of bit updates with optimal encoding for varying encoding lengths and word widths using the *Standard Scheme*, as explained in III-B. Note that this chart assumes $N/2$ bit updates from the encoding sequence.

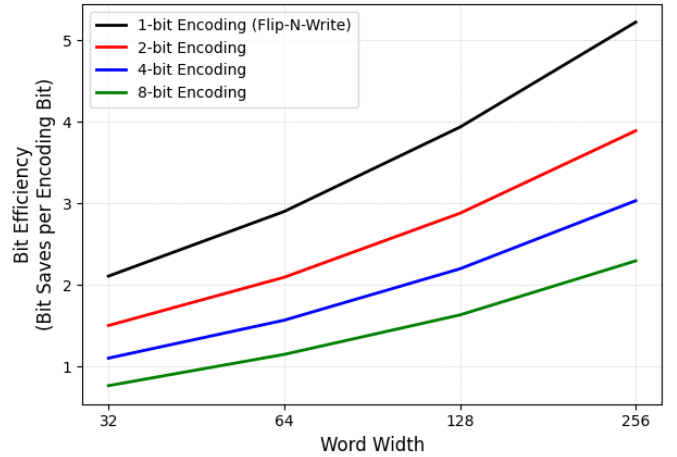


Fig. 3. Bit efficiency of encoding schemes, measured as the average number of bit transitions saved per encoding bit. Higher values indicate greater transition reduction per bit of encoding overhead.

TABLE I
MULTILAYER PERCEPTRON (MLP) ARCHITECTURE FOR ENCODING PREDICTION.

Layer	Type	Details
1	Dense (Hidden)	128 units, ReLU activation
2	Dense (Hidden)	64 units, ReLU activation
3	Dense (Hidden)	64 units, ReLU activation
4	Dense (Output)	N encoding units, Sigmoid activation
Loss Function: Binary Cross-Entropy Optimizer: Adam (learning rate = 0.001) Training: 30 epochs, batch size = 32		

D. Experimental Results

Fig. 5 shows the average number of bit updates for each inference model across varying encoding lengths and word

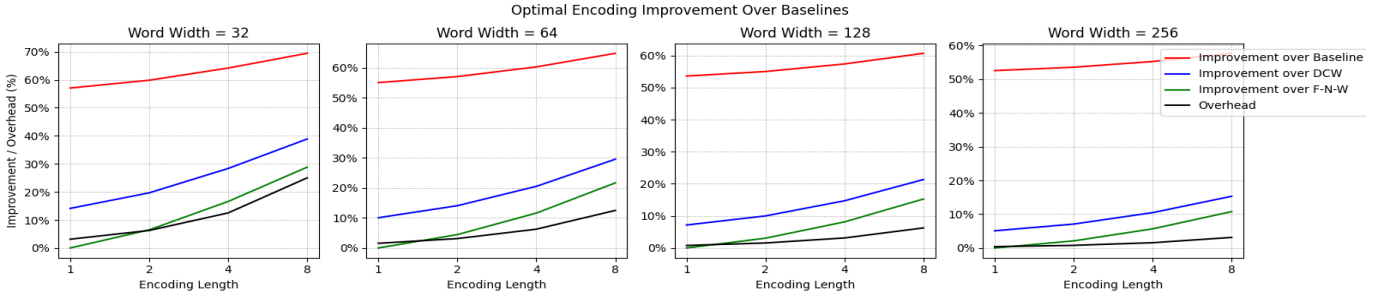


Fig. 4. Percentage improvement in bit-update reduction for optimal N -bit encoding schemes over baseline methods. Each subplot shows the improvement over raw writes (red), DCW (blue), and Flip-N-Write (green), alongside the encoding overhead (black), for increasing encoding lengths at fixed word widths.

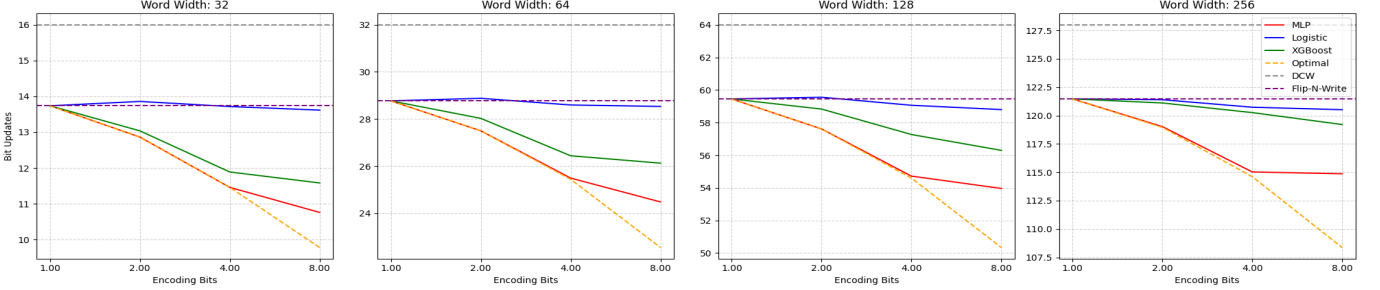


Fig. 5. Bit-update performance of learned inference models across varying encoding lengths and word widths. Results are plotted against the DCW, Flip-N-Write, and the optimal encoding in the standard scheme outlined in III-B. All results account for the encoding overhead $N/2$ bit updates per write.

widths. All results account for encoding overhead ($N/2$) to reflect the true cost of applying each scheme.

We include three reference baselines for comparison: DCW, Flip-N-Write, and the Optimal Encoding (computed via brute-force search).

Across all word lengths, the MLP performed the best, and saw consistent improvements until $N = 8$, where it began to fall short of the optimal mark. While larger MLPs provided even better accuracy, they were deemed impractically large for real time deployment.

XGBoost performs competitively for smaller word widths but loses effectiveness as word width increases. This may be due to underfitting.

Logistic Regression underperforms at all word lengths, likely due to its inability to model nonlinear relationships required to infer useful composite masks.

We tested XGBoost and Logistic regression again on a larger dataset (400,000 samples), which provided similar outcomes.

E. Trade-Offs

The results in Fig. 5 demonstrate that learned inference is an effective strategy for selecting encoding masks that minimize bit-level transitions during memory write operations. However, it is important to highlight the trade-offs associated with this approach.

The encoding overhead contributes, on average, $N/2$ bit updates per write. This becomes particularly significant for smaller word widths or when the benefit of encoding saturates.

Storage overhead is also an important metric to consider. As shown in Fig. 5, for a word width of 256 and encoding length of 8, only 3.1% overhead is required to enjoy $> 10\%$ *optimal* improvements over Flip-N-Write and 15% over DCW. Increasing overhead to 12.5% ($L = 64, N = 8$) further increases this margin to 21% and 29% respectively.

Model complexity also plays a critical role. Deeper MLP models achieve much higher accuracy, but require more memory and computational resources. Opting for a model such as XGBoost results in faster overall performance, but slightly worse encoding inference.

Additionally, these models require labeled training data, which becomes computationally expensive as encoding size increases.

V. IMPLEMENTATION DISCUSSION

Deploying this encoding strategy at the memory controller level imposes stringent constraints on both inference latency and storage overhead. Fast inference is essential to minimize write bottlenecks, while minimal storage overhead is necessary to preserve memory density.

The bit efficiency curves shown in Fig. 3 and the overhead curves displayed in Fig. 4 demonstrate a strong positive trend, where each encoding bit contributes more and costs less as word width increases, highlighting performance gains that scale with word width.

XGBoost, albeit fast at inference, does not scale to larger word widths as gracefully as the Multi-Layer Perceptron

(MLP), as shown in Fig. 5. MLPs, however, are generally more computationally expensive and take longer to execute.

As a direction of future work, Deep Differentiable Logic Gate Networks [22] provide a compelling alternative that allows for fast inference speed while simultaneously preserving modeling complexity comparable to MLPs.

VI. CONCLUSION

This paper introduces Learn2Mask, a predictive encoding framework for energy-efficient PCM writes. The proposed method utilizes a base set of modular affine sequences to generate structured bitmasks that effectively reduce bit-level transitions during memory writes. Experimental results confirm that these encodings lead to significant improvements in energy efficiency while maintaining low system overhead.

We further demonstrate that lightweight machine learning models, such as XGBoost and MLPs, can approximate optimal encodings with high accuracy. This enables real-time, scalable deployment of Learn2Mask in memory systems, offering a practical and intelligent solution for optimizing write operations in PCM.

Looking ahead, several promising directions can further enhance the capabilities of the Learn2Mask framework. One avenue involves expanding beyond the current affine pattern space to incorporate more expressive basis sets or dynamically learned mask families, potentially utilizing evolutionary strategies or deep search techniques. Another area of interest is the integration of reinforcement learning, where online agents could adapt encoding strategies in response to changing workload patterns, enabling more flexible and intelligent behavior. Additionally, developing heuristic or approximate methods for label generation could help overcome the computational limitations of brute-force search, especially for larger encoding lengths. Hardware prototyping remains a critical next step, where implementing Learn2Mask within a real PCM memory controller would allow for practical evaluation of latency, area, and energy overheads. Finally, future studies should investigate the framework's robustness under biased or imbalanced data distributions, and assess whether adaptive retraining or domain adaptation techniques are needed to maintain model accuracy in diverse operating conditions. Together, these directions pave the way for deploying Learn2Mask as a scalable and efficient solution in next-generation non-volatile memory systems.

VII. ACKNOWLEDGMENT

This work was supported by the National Science Foundation (NSF) under Grant Nos.[CNS-2244391] and [2318553]. We gratefully acknowledge the NSF for their financial support.

REFERENCES

- [1] J. P. S. Tharanyaa, D. Sharmila, and R. S. Kumar, "A novel workload-aware and optimized write cycles in nvram," *Computers, Materials & Continua*, vol. 71, no. 2, pp. 2667–2681, 2022.
- [2] K. Fatemi and Masoud, "A power-aware reinforcement learning technique for memory allocation in real-time embedded systems." <https://prism.ucalgary.ca>, 2022. Available at: <https://prism.ucalgary.ca>.
- [3] M. Shi, P. Mo, and J. Liu, "Deep neural network for accurate and efficient atomistic modeling of phase change memory," *IEEE Electron Device Letters*, vol. 41, pp. 365–368, Mar. 2020.
- [4] Z. Chen and D. Marculescu, "Distributed reinforcement learning for power limited many-core system performance optimization," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Jan. 2015.
- [5] S. Song, A. Das, O. Mutlu, and N. Kandasamy, "Improving phase change memory performance with data content aware access," *arXiv preprint arXiv:2006.10991*, June 2020.
- [6] Z. Wang and et al., "Modular reinforcement learning for self-adaptive energy efficiency optimization in multicore system," in *ASP-DAC '17: Asia and South Pacific Design Automation Conference*, 2017.
- [7] Z. Deng, L. Zhang, N. Mishra, H. Hoffmann, and F. T. Chong, "Memory cocktail therapy: A general learning-based framework to optimize dynamic tradeoffs in nvms," in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017.
- [8] M. Kim, H. Lee, H. Kim, and H.-J. Lee, "W1-wd: Wear-leveling solution to mitigate write disturbance errors for phase-change memory," *IEEE Access*, vol. 10, pp. 11420–11431, 2022.
- [9] J. Jang, W. Shin, J. Choi, Y. Kim, and L.-S. Kim, "Sparse-insertion write cache to mitigate write disturbance errors in phase change memory," *IEEE Transactions on Computers*, vol. 68, no. 5, pp. 752–764, 2019.
- [10] S. M. Seyedzadeh, A. K. Jones, and R. Melhem, "Improving sustainability through disturbance crosstalk mitigation in deeply scaled phase-change memory," in *2018 Ninth International Green and Sustainable Computing Conference (IGSC)*, pp. 1–8, 2018.
- [11] Y. Lim, D. Kim, and J. Kim, "Sec: Efficient error correction codes for mlc pcm," in *2023 20th International SoC Design Conference (ISOCC)*, pp. 303–304, 2023.
- [12] M. Desai, A. Rumale, M. Asadinia, and S. Bogle, "Wire: Write energy reduction via encoding in phase change main memories (pcm)," in *Proceedings of the Future Technologies Conference (FTC) 2024, Volume 3* (K. Arai, ed.), vol. 1156 of *Lecture Notes in Networks and Systems*, pp. 641–655, Springer, Cham, 2024.
- [13] B.-D. Yang, J.-E. Lee, J.-S. Kim, J. Cho, S.-Y. Lee, and B.-G. Yu, "A low power phase-change random access memory using a data-comparison write scheme," in *2007 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 3014–3017, 2007.
- [14] S. Cho and H. Lee, "Flip-n-write: A simple deterministic technique to improve pram write performance, energy and endurance," in *2009 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 347–357, 2009.
- [15] S. M. Seyedzadeh, R. Maddah, D. Kline, A. K. Jones, and R. Melhem, "Improving bit flip reduction for biased and random data," *IEEE Transactions on Computers*, vol. 65, no. 11, pp. 3345–3356, 2016.
- [16] M. Poremba, T. Zhang, and Y. Xie, "Nvmmain 2.0: A user-friendly memory simulator to model (non-)volatile memory systems," *IEEE Computer Architecture Letters*, vol. 14, pp. 140–143, July 2015.
- [17] M. Poremba and Y. Xie, "Nvmmain: An architectural-level main memory simulator for emerging non-volatile memories," in *Proceedings of the 2012 IEEE Computer Society Annual Symposium on VLSI, ISVLSI '12*, (USA), p. 392–397, IEEE Computer Society, 2012.
- [18] M. Desai, R. Quinn, and M. Asadinia, "Smart-write: Adaptive learning-based write energy optimization for phase change memory," in *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 00640–00648, 2025.
- [19] J. Ekoniak, A. Rumale, and M. Asadinia, "MI-prep: Machine learning based error prediction for phase change memory," in *2025 IEEE 15th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 00909–00917, 2025.
- [20] C. D. Spradling, "Spec cpu2006 benchmark tools," *SIGARCH Comput. Archit. News*, vol. 35, p. 130–134, Mar. 2007.
- [21] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: characterization and architectural implications," in *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques, PACT '08*, (New York, NY, USA), p. 72–81, Association for Computing Machinery, 2008.
- [22] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, "Deep differentiable logic gate networks," in *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, (Red Hook, NY, USA), Curran Associates Inc., 2022.